

Sistemas Distribuídos

Programação com Sockets

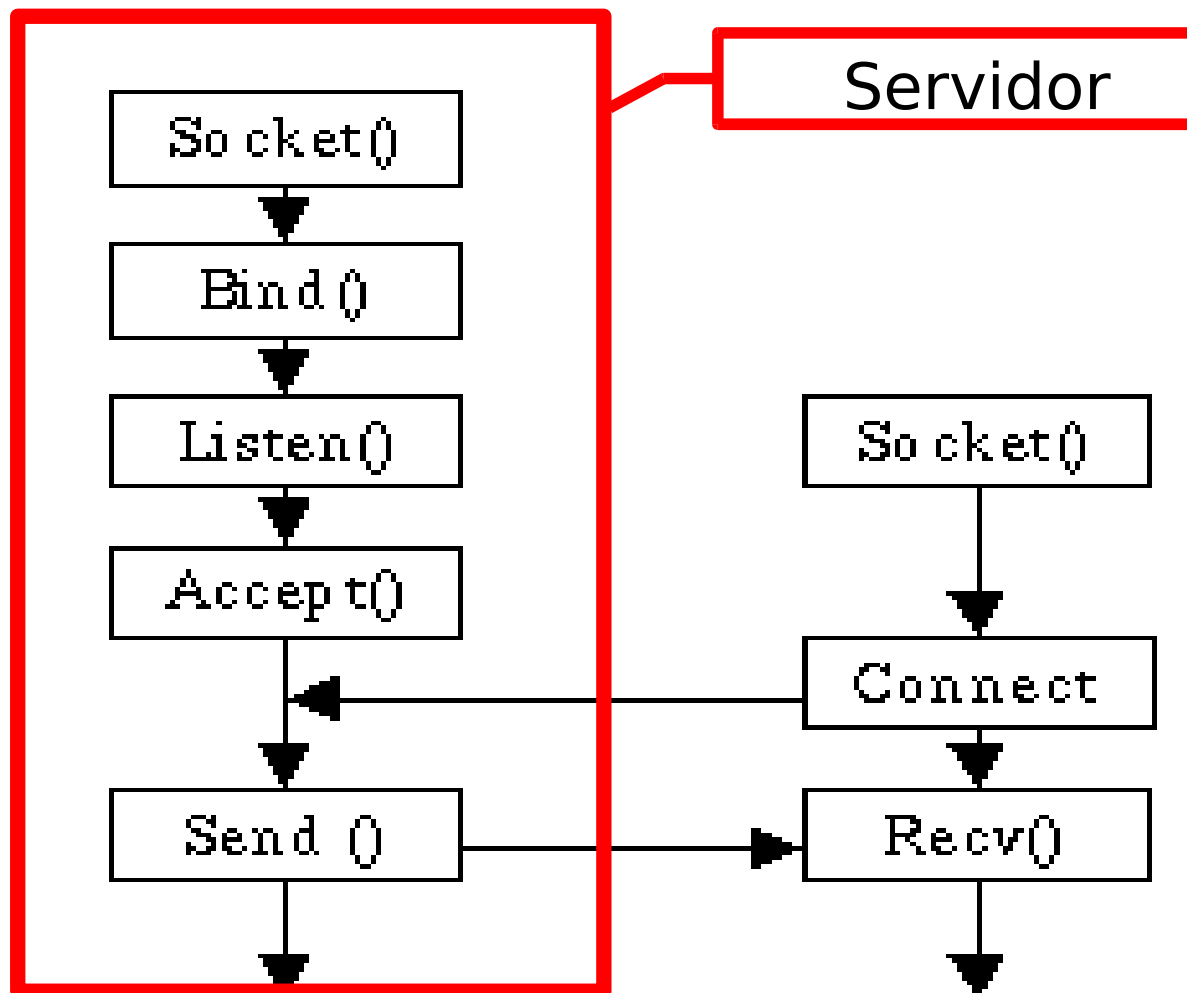
André Luis Martinotto

Programação com Sockets TCP/IP

Objetivos

- Criar uma aplicação do tipo Cliente-Servidor com conexão.
- O Servidor periodicamente (por exemplo a cada minuto) envia a sua hora local para o Cliente.
- O servidor deve suportar múltiplos clientes ao mesmo tempo.

Modelo TCP/IP

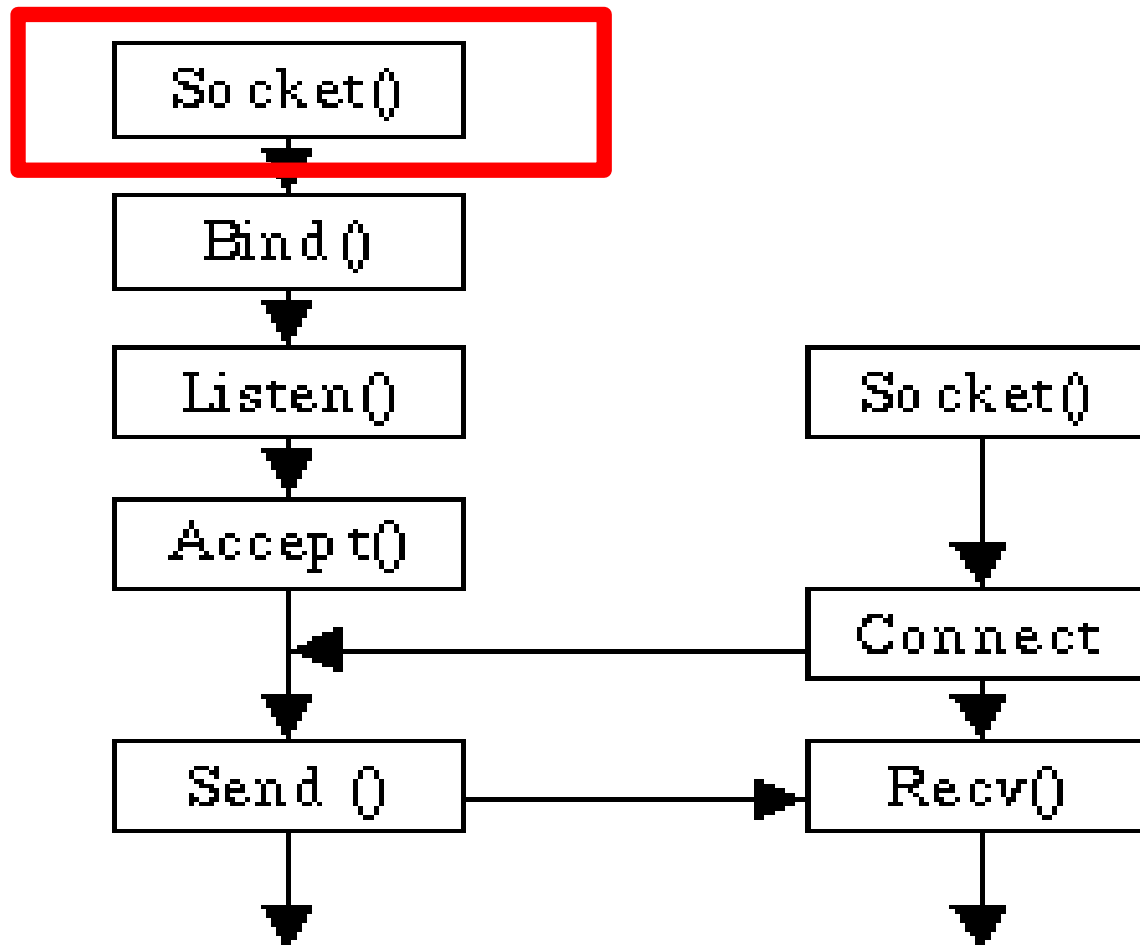


Bibliotecas Necessárias e Definições

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

#define CONEXOES 5
```

Servidor – Criando o Socket



Servidor – Criando o Socket

```
void main(int argc, char **argv){
```

```
    struct sockaddr_in servidor;
```

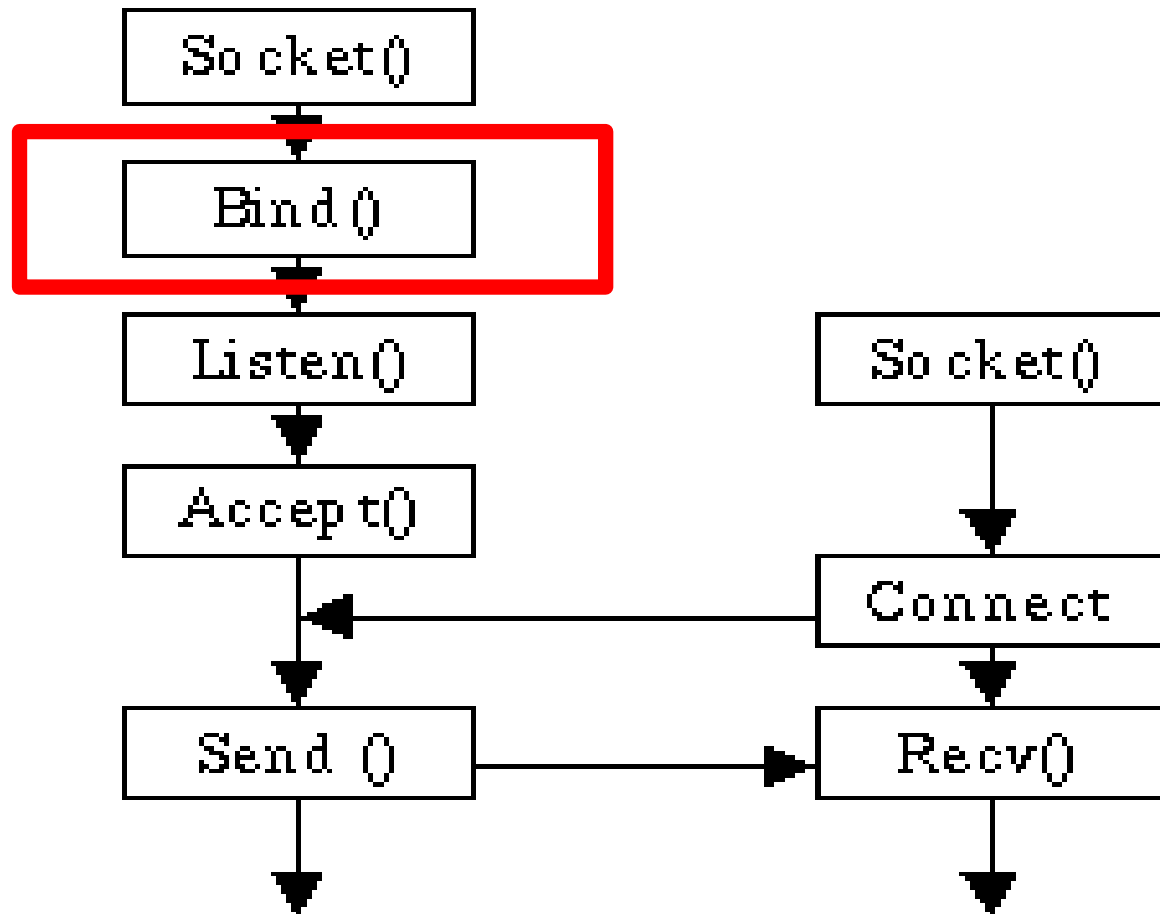
```
    int soquete_servidor;
```

```
    int soquete_cliente;
```

```
    int pid;
```

```
    soquete_servidor=socket  
        (AF_INET,SOCK_STREAM,0);
```

Servidor – Atribuindo uma Porta

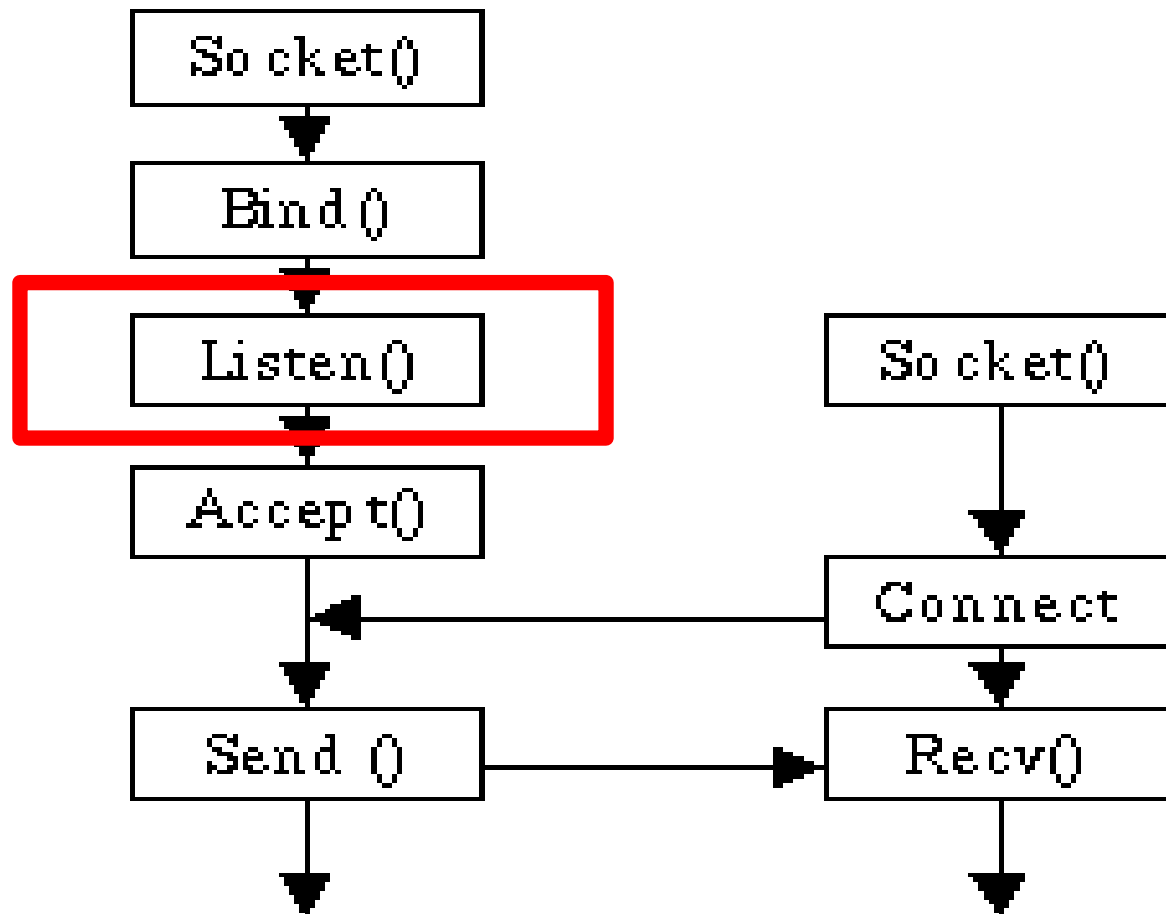


Servidor – Atribuindo uma Porta

```
bzero((char *)&servidor,sizeof(servidor));  
servidor.sin_family = AF_INET;  
servidor.sin_port = htons(atoi(argv[1]));  
servidor.sin_addr.s_addr = INADDR_ANY;
```

```
bind (soquete_servidor, (struct sockaddr *)&servidor,  
      sizeof(servidor));
```

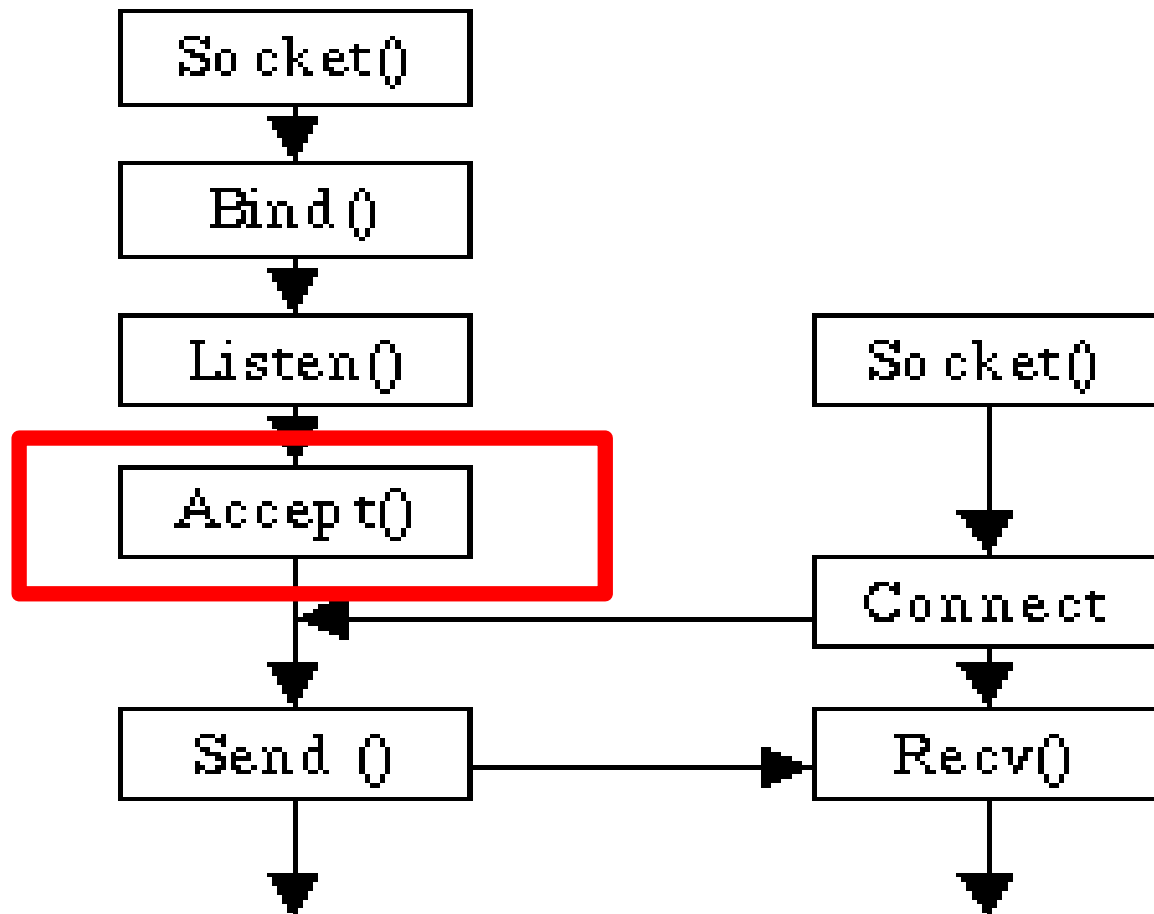
Servidor – Definindo uma Lista de Requisições



Servidor – Definindo uma Lista de Requisições

```
listen (soquete_servidor, CONEXOES);
```

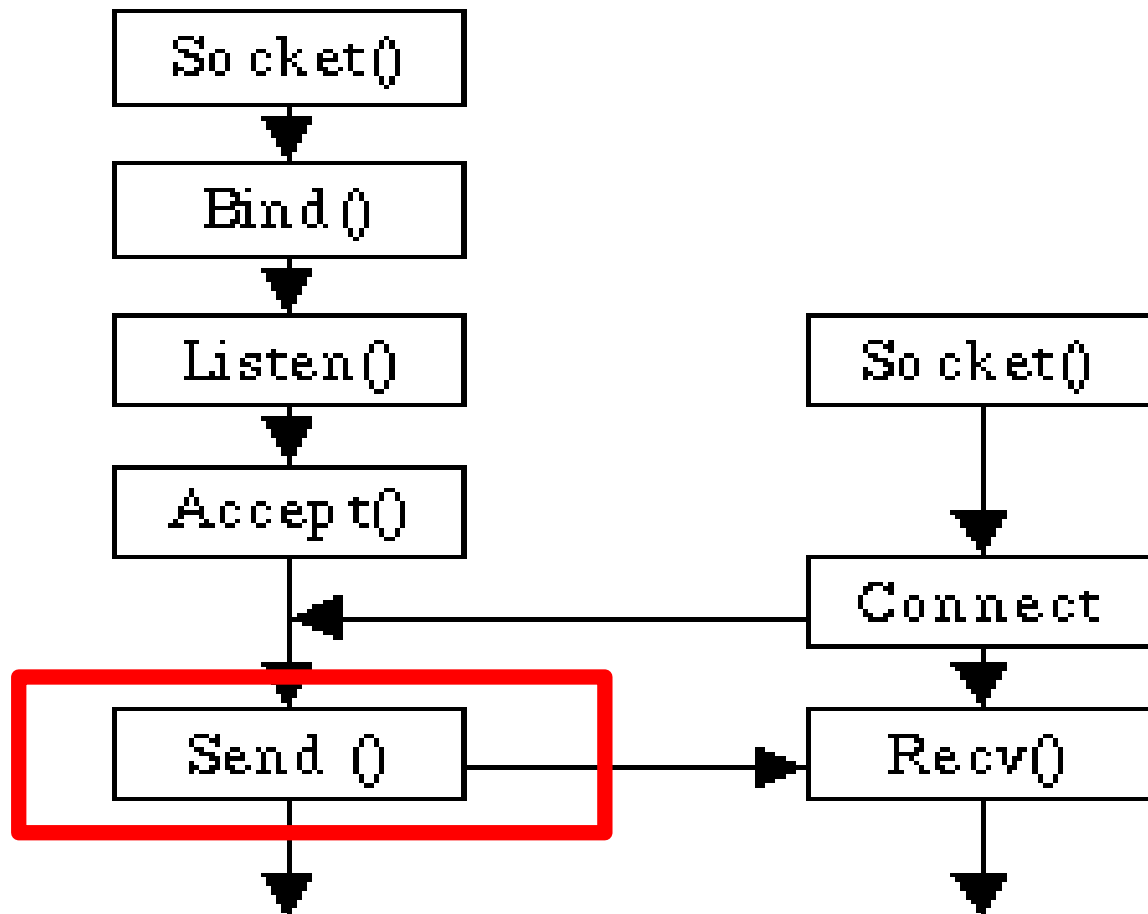
Servidor – Esperando uma Conexão



Servidor - Esperando uma Conexão

```
while (1){  
    soquete_cliente = accept (soquete_servidor,  
        NULL , NULL);  
  
    funcaoTempo(soquete_cliente);  
}  
}
```

Servidor – Esperando uma Conexão



Servidor – Enviando uma Mensagem

```
void funcaoTempo(int soquete){  
    time_t tempo;  
    char *str;  
  
    while ( 1 ){  
        tempo = time(NULL);  
        str= (char *) ctime(&tempo);  
        send(soquete,str,strlen(str)+1,0);  
        sleep(60);  
    }  
}
```

Compilando e Executando o Servidor

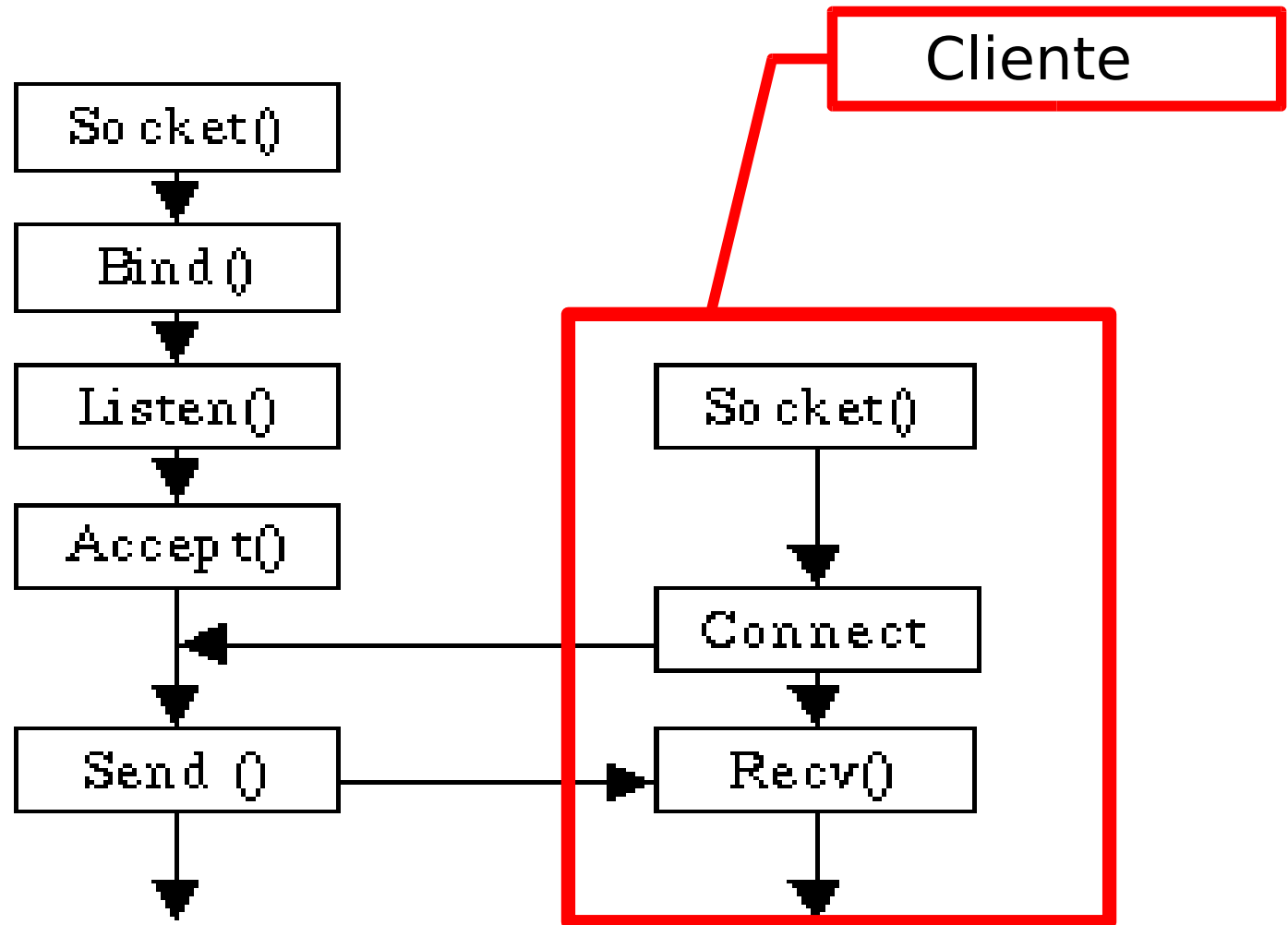
- Compilar:

```
gcc servidor.c -o servidor
```

- Executando:

```
./servidor <porta>
```


Modelo TCP/IP

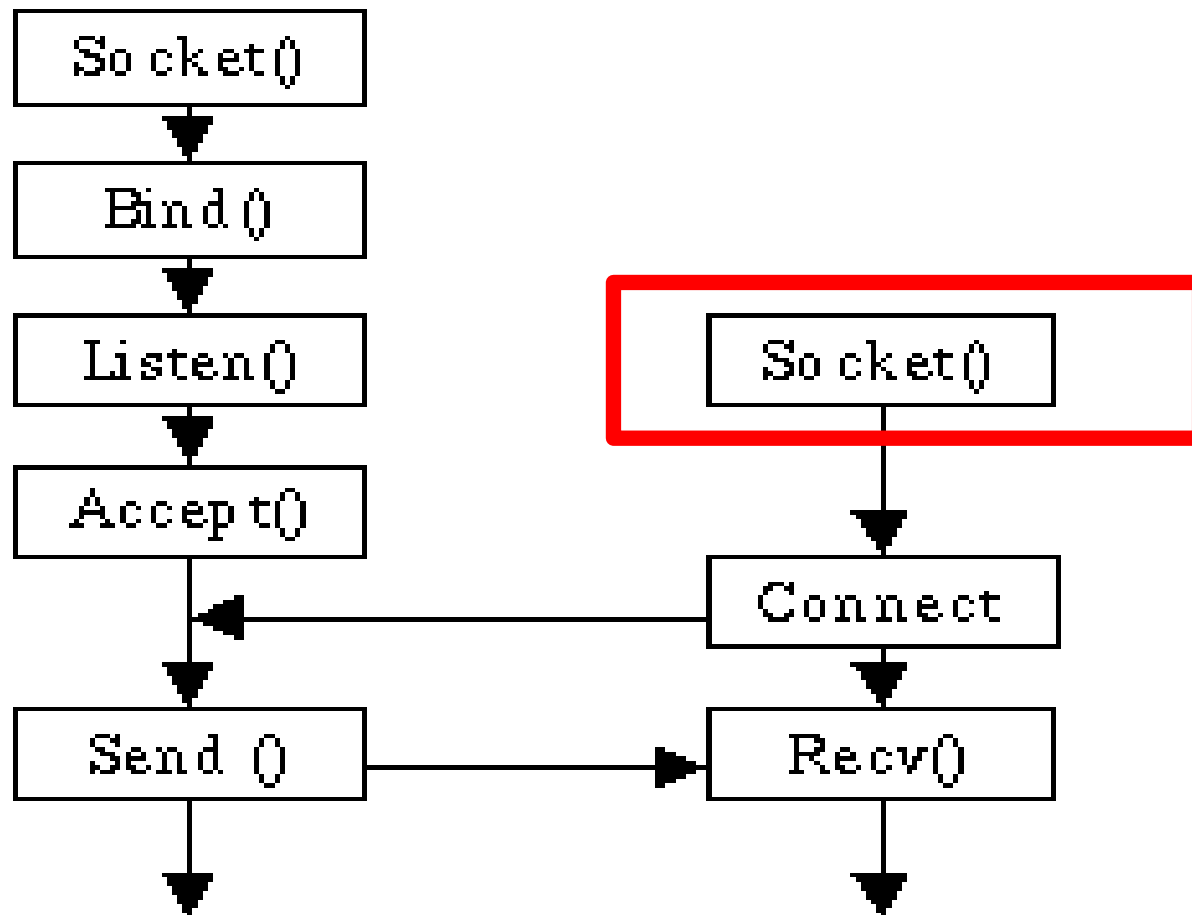


Bibliotecas Necessárias e Definições

```
#include <stdio.h>  
#include <sys/types.h>  
#include <sys/socket.h>  
#include <netinet/in.h>  
#include <stdlib.h>  
#include <string.h>
```

```
#define TAM 50
```

Cliente – Criando o Socket

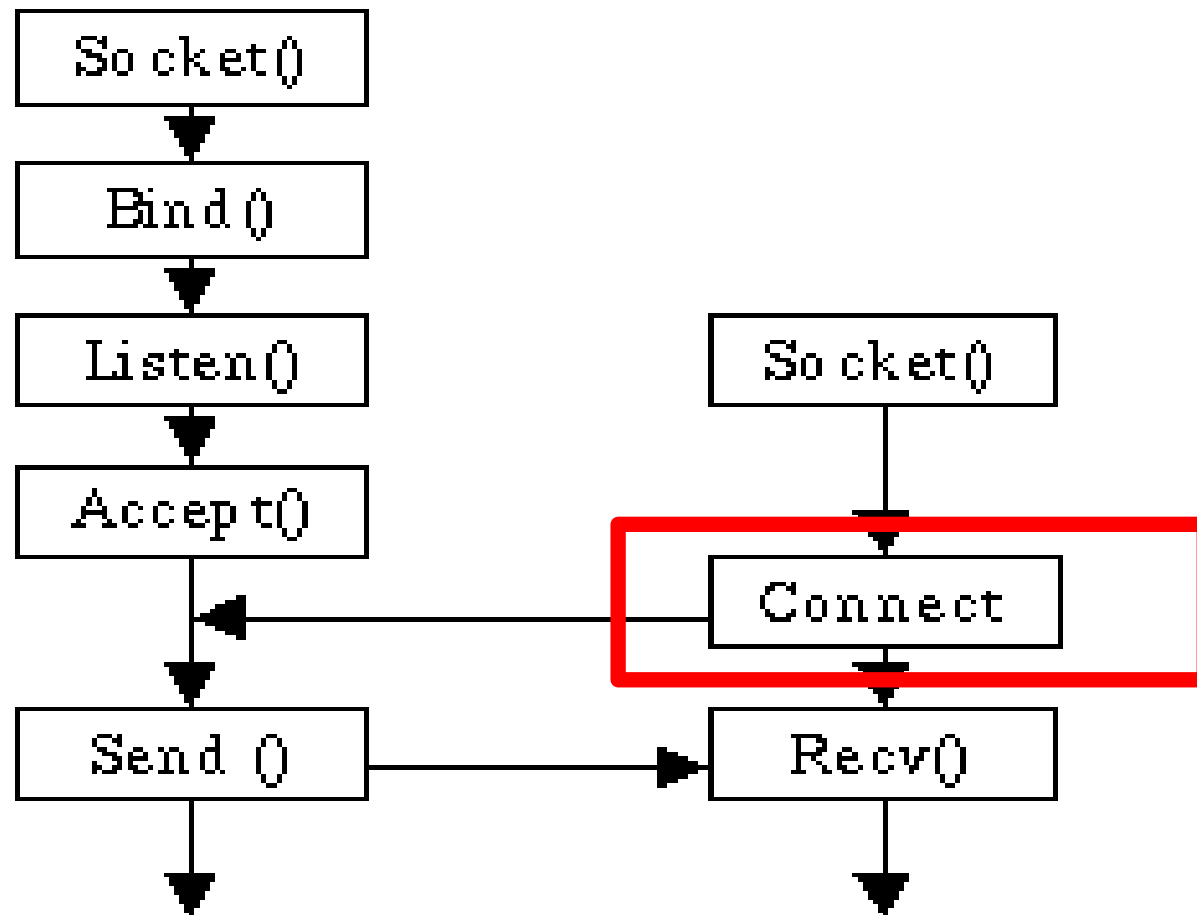


Cliente – Criando o Socket

```
void main(int argc, char **argv){  
    struct sockaddr_in servidor;  
    int soquete;  
    char str[TAM];
```

```
    soquete = socket (AF_INET,SOCK_STREAM,0);
```

Cliente – Criando o Socket

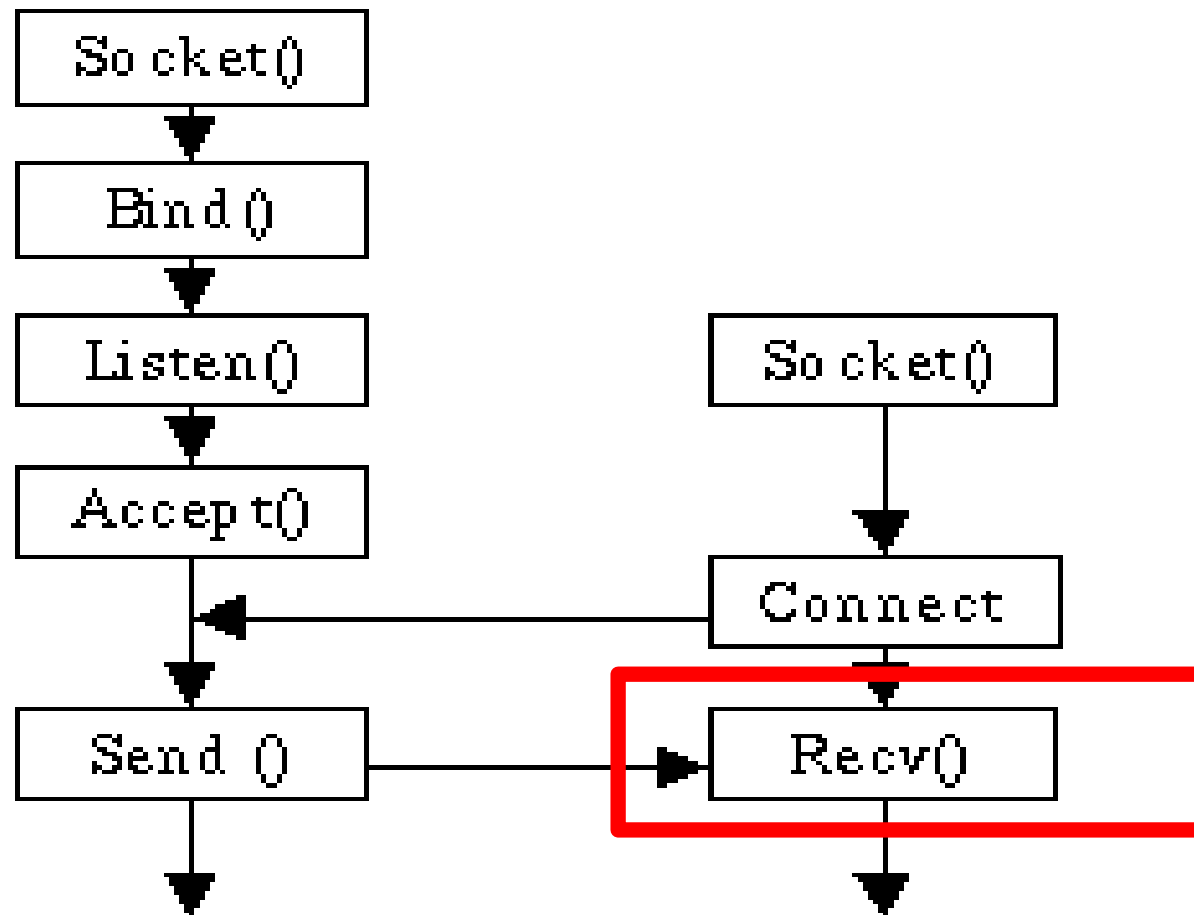


Cliente – Conectando ao Servidor

```
bzero((char *)&servidor,sizeof(servidor));  
servidor.sin_family = AF_INET;  
servidor.sin_port = htons(atoi(argv[2]));  
servidor.sin_addr.s_addr = inet_addr(argv[1]);
```

```
connect (soquete, (struct sockaddr *) &servidor,  
        sizeof(servidor));
```

Cliente – Criando o Socket



Cliente – Recebendo uma Mensagem

```
while ( 1 ){  
    recv(soquete, str, TAM, 0);  
    printf("TEMPO: %s",str);  
}  
}
```


Compilando e Executando o Servidor

- Compilar:

```
gcc cliente.c -o cliente
```

- Executando:

```
./cliente <host> <porta>
```

Suporte a Múltiplos Clientes

- Mas o servidor não aceita múltiplos Clientes;
- Solução:
 - A primitiva `accept` cria um novo *socket* com endereço do *socket* do cliente
 - O *socket* original continua como estava anteriormente.

```
soquete_cliente = accept (soquete_servidor, NULL ,  
NULL);
```

- Após a criação do *socket* cliente (`accept`) cria-se um nova *thread* ou processo para comunicação com o cliente
- O servidor pode continuar aceitando conexões no *socket* original.

Suporte a Múltiplos Clientes

```
while (1){
```

```
    soquete_cliente = accept (soquete_servidor, NULL ,  
        NULL);
```

```
    pid = fork();
```

```
    if ( pid > 0)  
        close(soquete_cliente);  
    else funcaoTempo(soquete_cliente)
```

```
}
```

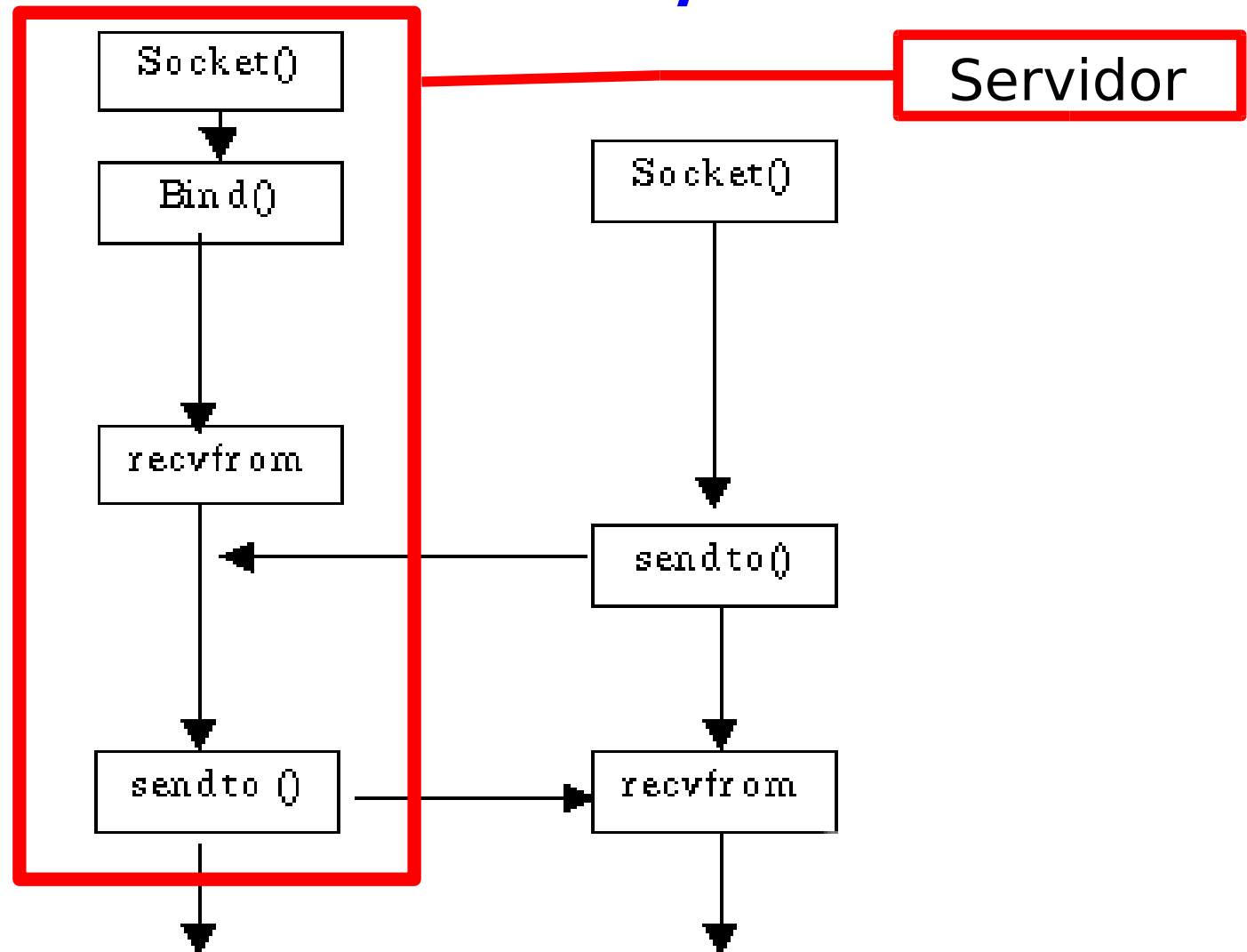
```
}
```

Programação com Sockets UDP/IP

Objetivos

- Criar uma aplicação do tipo Cliente-Servidor sem conexão.
- O Cliente envia uma mensagem texto para o servidor;
- O servidor recebe a mensagem e simplesmente a devolve ao cliente (concatenando a string “ :ECHO: ” no final da mensagem).

Modelo UDP/IP

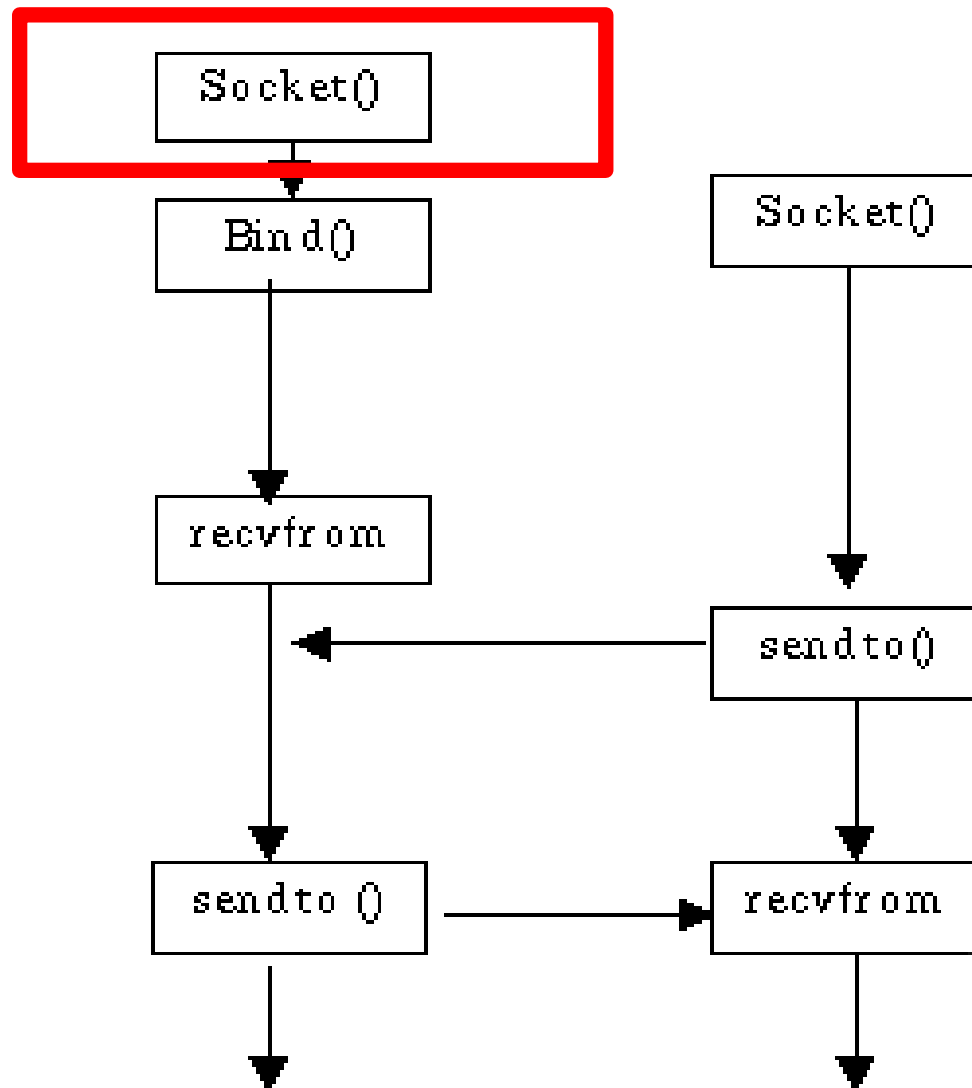


Bibliotecas Necessárias e Definições

```
#include <stdio.h>  
#include <sys/types.h>  
#include <sys/socket.h>  
#include <netinet/in.h>  
#include <stdlib.h>  
#include <string.h>
```

```
#define MENSAGEM 60
```

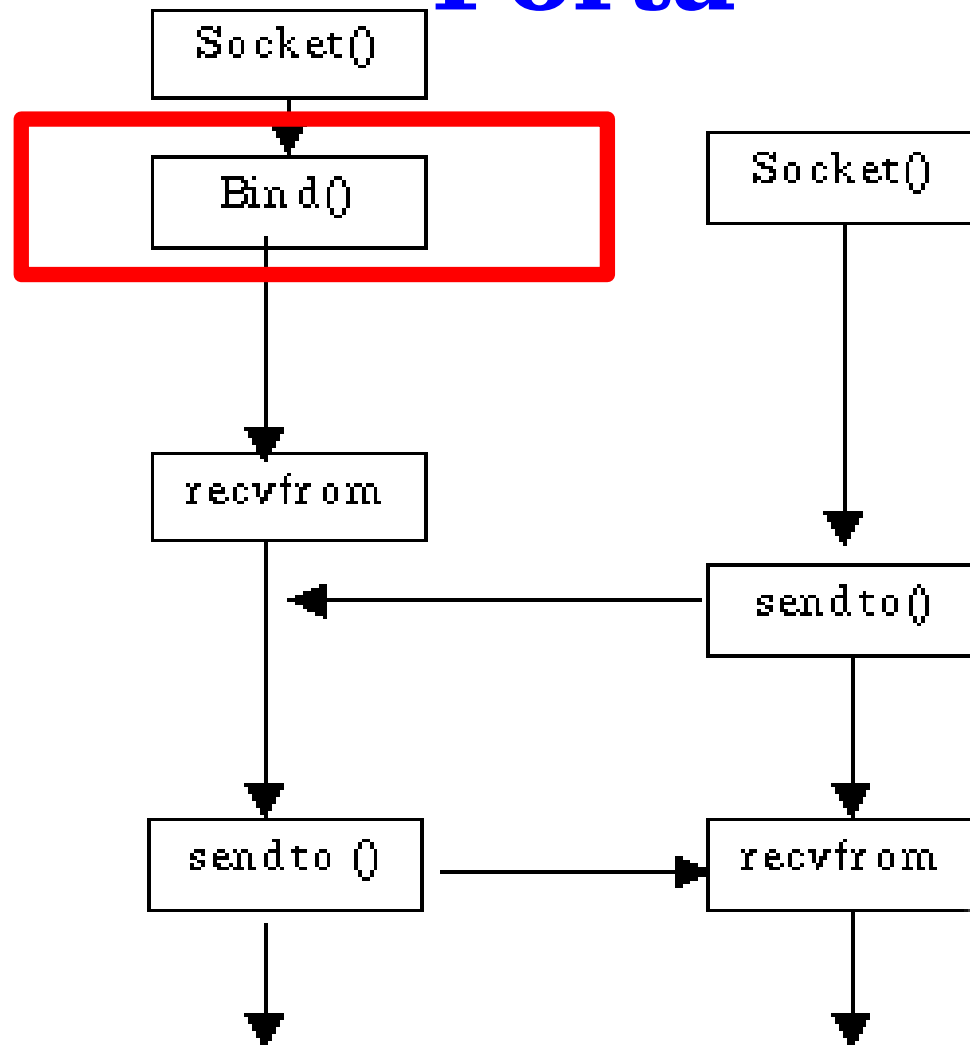
Servidor – Criando o Socket



Servidor – Criando o Socket

```
void main(int argc, char **argv){  
  
    struct sockaddr_in endereco_servidor;  
    struct sockaddr_in endereco_cliente;  
    int soquete_servidor;  
    int soquete_cliente;  
    int tamanho;  
    char string[MENSAGEM];  
  
    soquete_servidor = socket (AF_INET,SOCK_DGRAM,0);
```

Servidor – Atribuindo uma Porta



Servidor - Atribuindo uma Porta

```
bzero((char*)&endereco_servidor,sizeof(endereco_servidor));
```

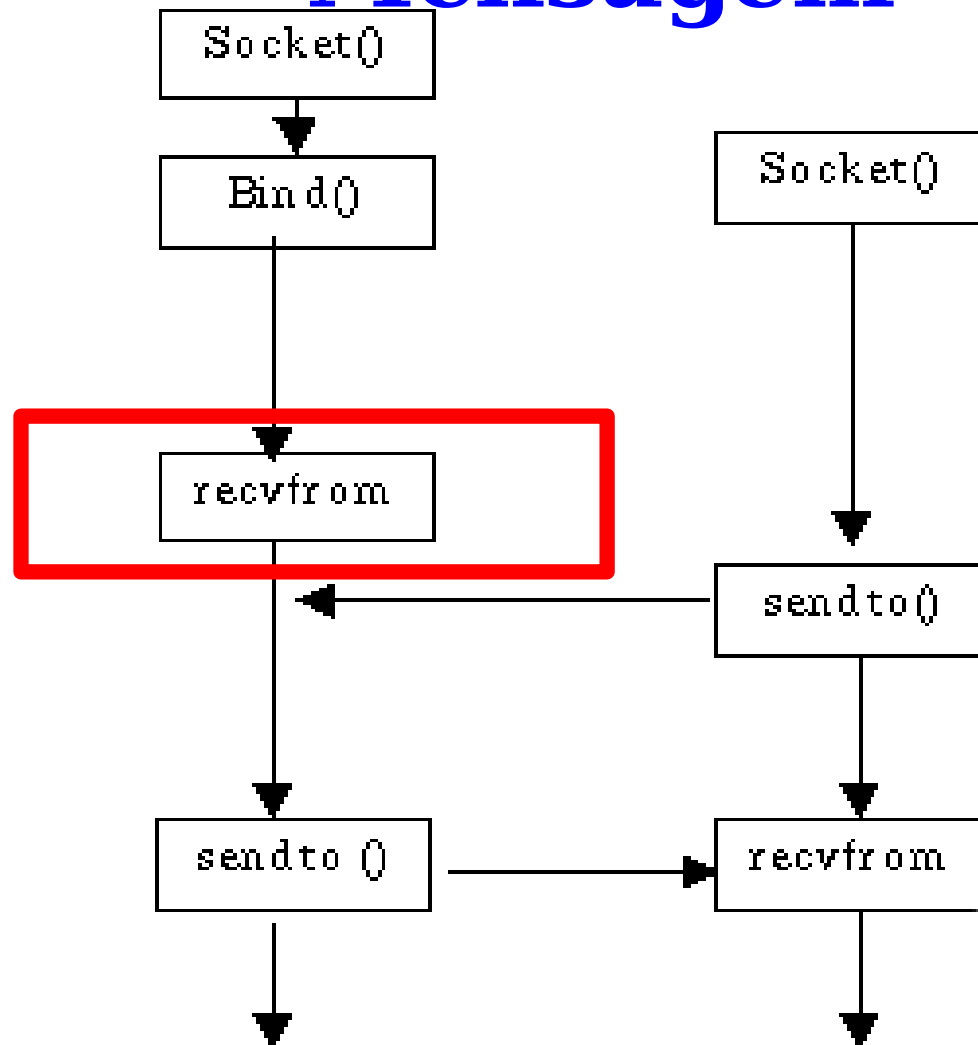
```
endereco_servidor.sin_family = AF_INET;
```

```
endereco_servidor.sin_port = htons(atoi(argv[1]));
```

```
endereco_servidor.sin_addr.s_addr = INADDR_ANY;
```

```
bind (soquete_servidor, (struct sockaddr  
    *)&endereco_servidor, sizeof(endereco_servidor));
```

Servidor-Recebendo uma Mensagem



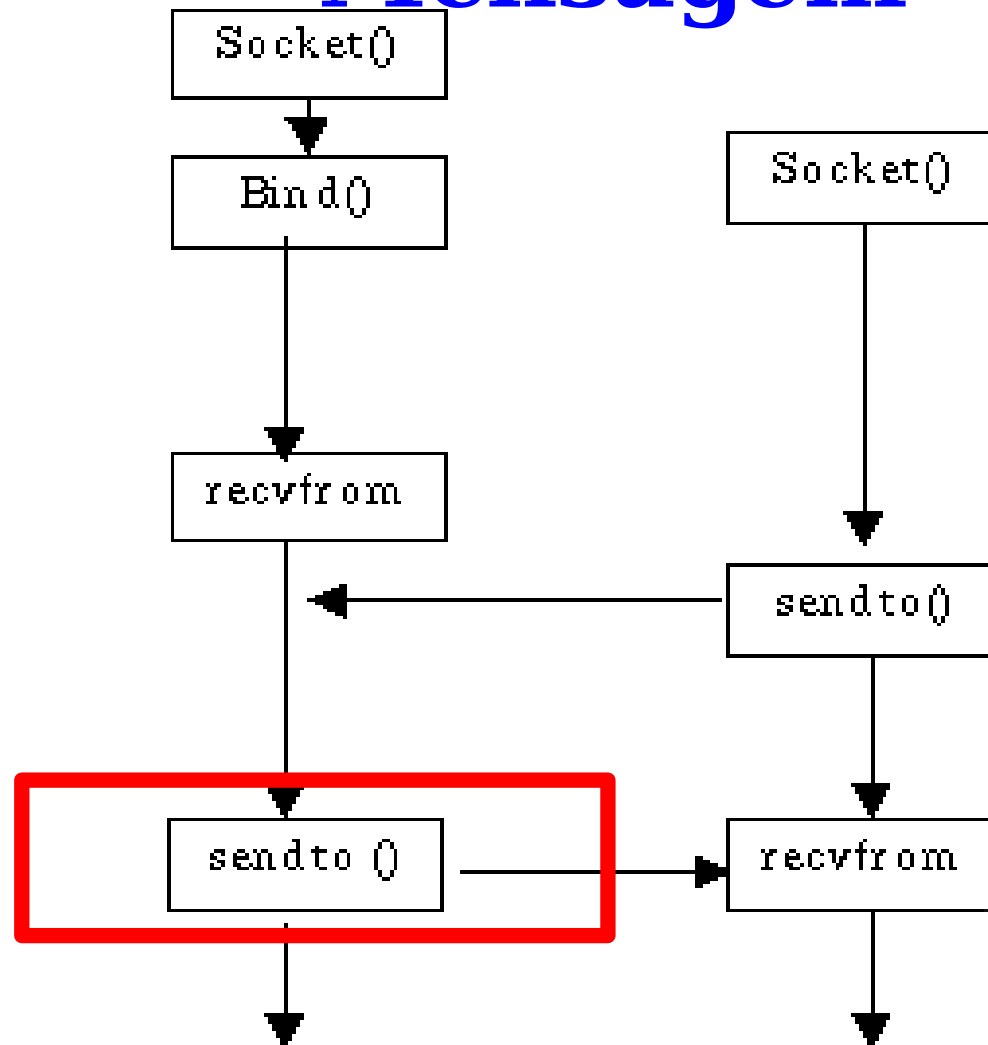
Servidor-Recebendo uma Mensagem

```
while (1){
```

```
    tamanho = sizeof(endereco_cliente);
```

```
    recvfrom (soquete_servidor, string,  
             MENSAGEM*sizeof(char), 0, (struct sockaddr  
             *)&endereco_cliente, &tamanho);
```

Servidor - Enviando uma Mensagem



Servidor – Enviando uma Mensagem

```
strcat (string, " :ECHO: ");
```

```
sendto ( soquete_servidor, string,  
        (strlen(string)+1)*sizeof(char), 0, (struct  
        sockaddr *)&endereco_cliente,  
        sizeof(endereco_cliente));
```

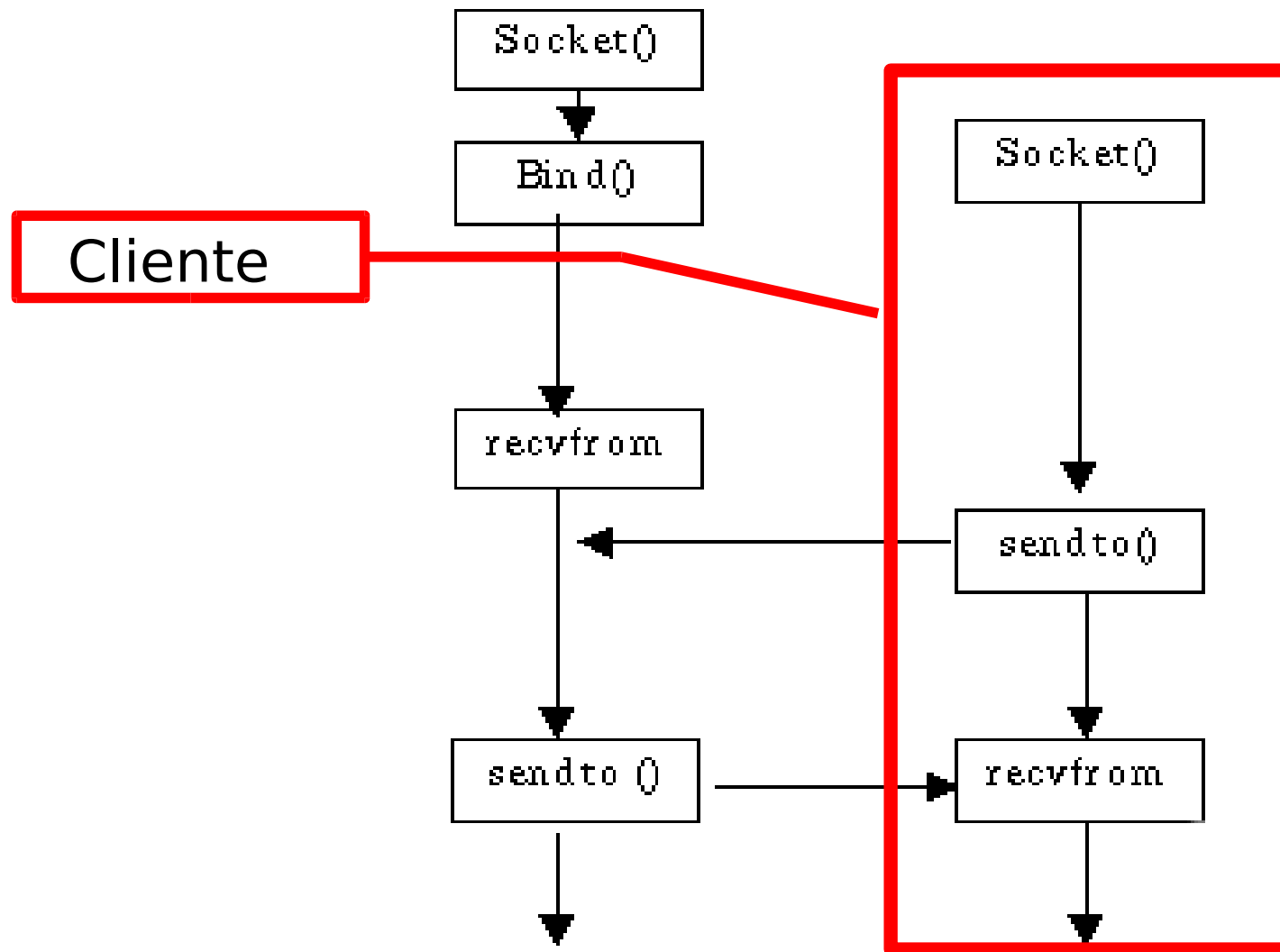
```
}
```

Servidor – Fechando o Socket

```
close(soquete_servidor);
```

```
}
```


Modelo UDP/IP

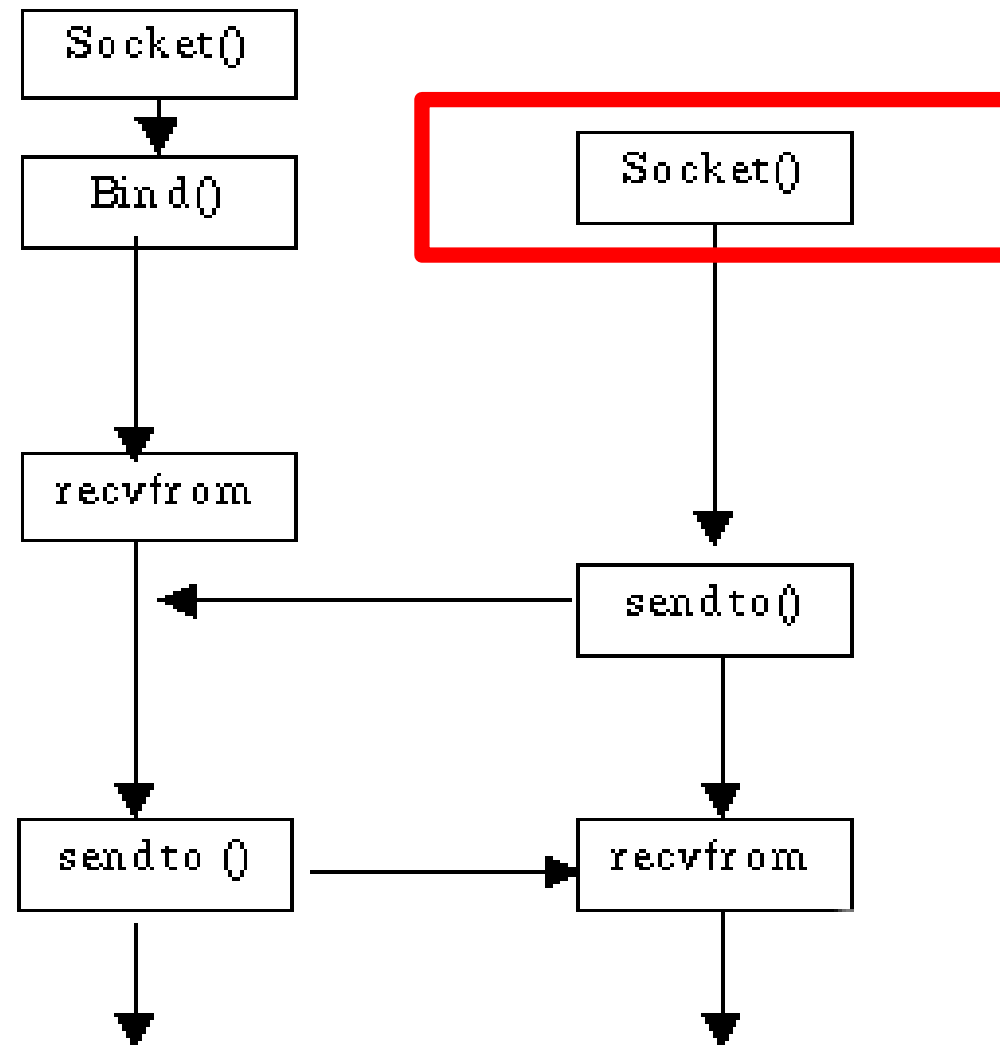


Bibliotecas Necessárias e Definições

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <string.h>

#define MENSAGEM 60
```

Cliente – Criando o Socket

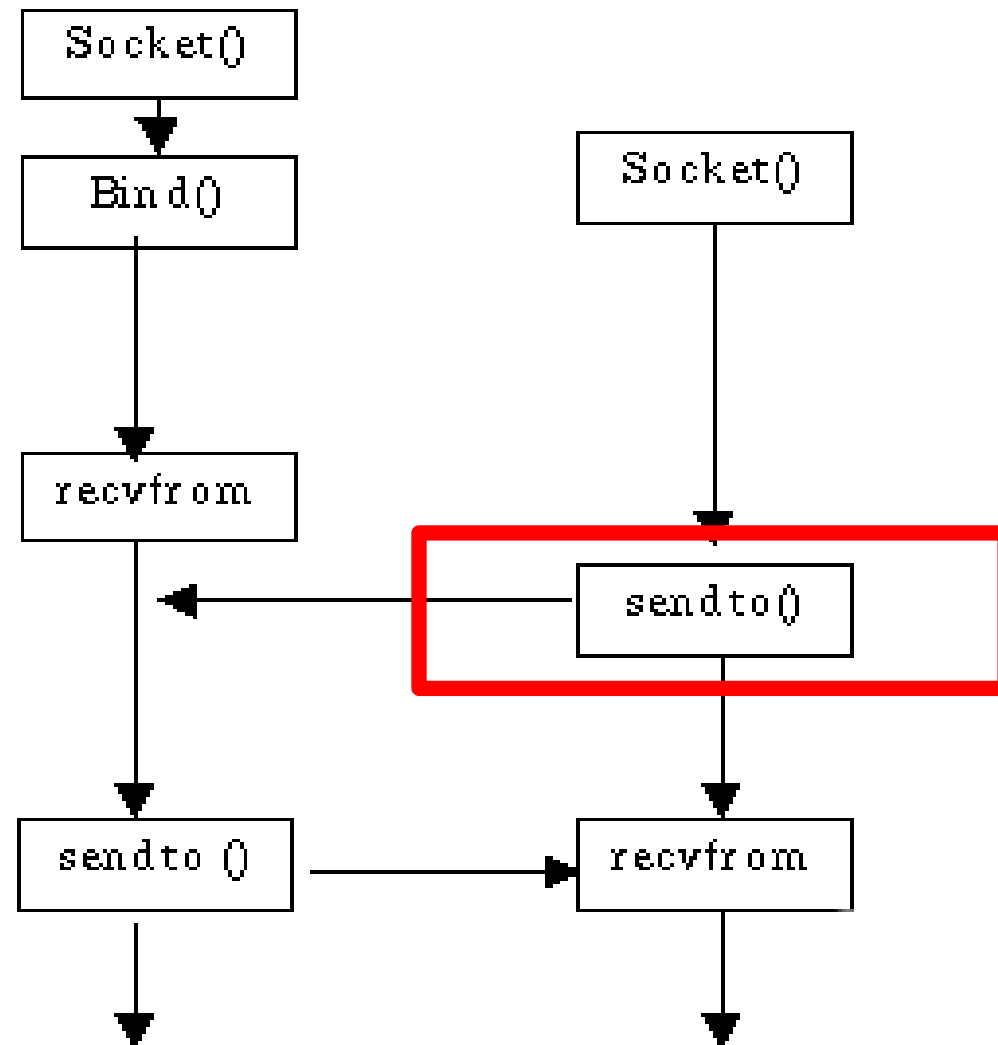


Cliente – Criando o Socket

```
void main(int argc, char **argv){  
  
    struct sockaddr_in endereco_servidor;  
    int soquete;  
    char string[MENSAGEM];
```

```
    soquete = socket (AF_INET,SOCK_DGRAM,0);
```

Cliente – Enviando uma Mensagem

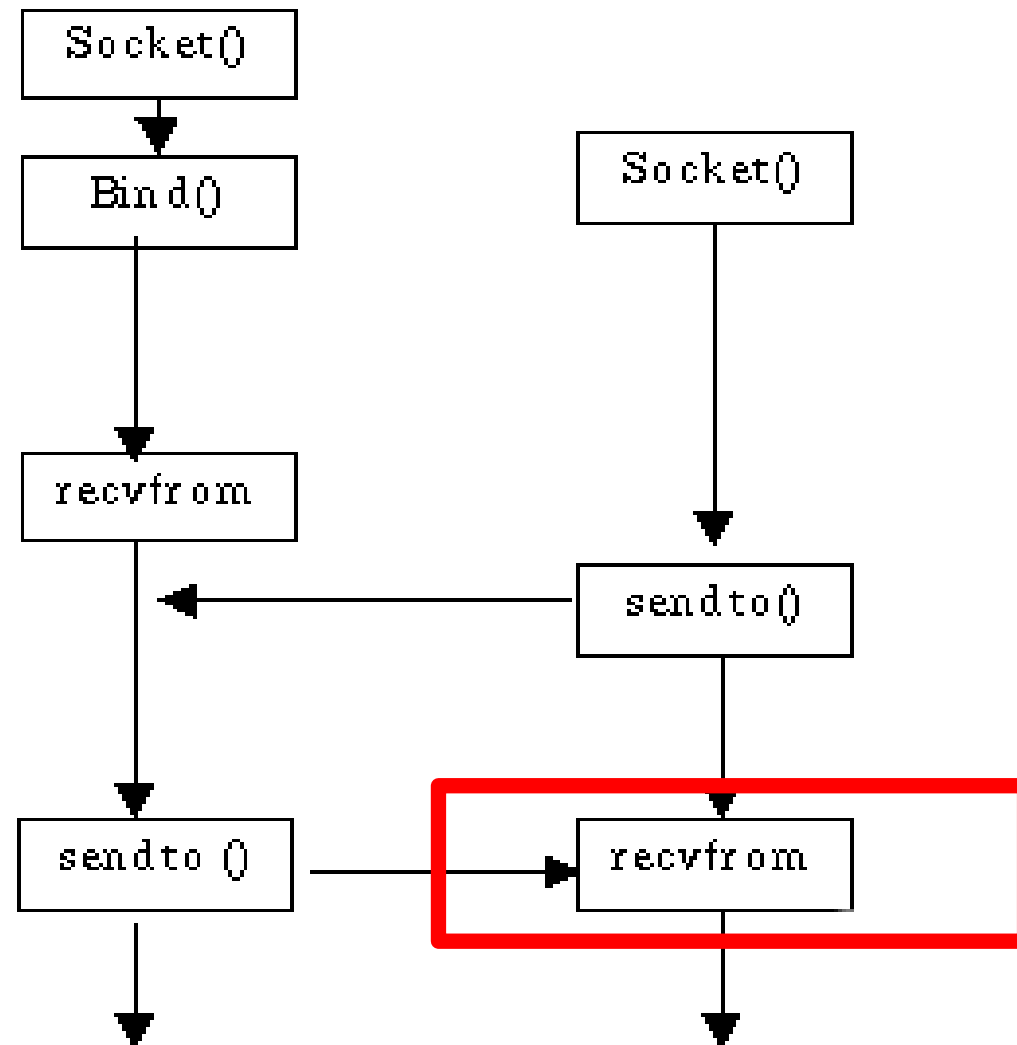


Cliente – Enviando uma Mensagem

```
bzero((char  
    *)&endereco_servidor,sizeof(endereco_servidor));  
endereco_servidor.sin_family = AF_INET;  
endereco_servidor.sin_port = htons(atoi(argv[2]));  
endereco_servidor.sin_addr.s_addr = inet_addr(argv[1]);  
  
strcpy (string, "HELLO");
```

```
sendto (soquete, string, (strlen(string)+1)*sizeof(char),  
    0, (struct sockaddr  
    *)&endereco_servidor,sizeof(endereco_servidor));
```

Cliente – Recebendo uma Mensagem



Cliente – Recebendo uma Mensagem

```
recvfrom (soquete, string,  
          MENSAGEM*sizeof(char), 0, NULL, NULL );
```

```
printf("%s\n",string);
```


Cliente – Fechando o Socket

```
close(soquete);
```

```
}
```